Black Box API Testing Report

QuickCart REST API

1. Overview

System Under Test	QuickCart REST API (quickcart:latest, x86 Docker image)
Tools Used	pytest, requests
Test Results	82 passed | 16 failed | 20 skipped

---

2. Test Case Documentation

Each test case documents the input, expected output, and a justification explaining why the case is necessary for validating the API's behaviour. Cases that uncovered confirmed server bugs are labelled accordingly.

Module 1: Authentication Headers (test_auth.py)

These tests verify that the API correctly enforces the X-Roll-Number and X-User-ID header requirements across all endpoints.

#	Test Case	Input	Expected	Justification
1	Missing X-Roll-Number	No headers	401	Per the spec, a missing roll number must return 401 Unauthorized.
2	Letters in X-Roll-Number	X-Roll-Number: abc	400	The roll number must be an integer; non-integer values must be rejected with 400.
3	Symbols in X-Roll-Number	X-Roll-Number: !@#	400	Special characters are not valid integers and must be rejected.
4	Valid X-Roll-Number	X-Roll-Number: 2025121002	200	A correctly formatted roll number must be accepted and return 200.
5	Missing X-User-ID on user endpoint	Only X-Roll-Number on /profile	400	Spec states that a missing X-User-ID on user endpoints must return 400.
6	X-User-ID = 0	X-User-ID: 0	400	The user ID must be a positive integer; zero is not a valid value.

#	Test Case	Input	Expected	Justification
7	Non-existent X-User-ID (999999)	X-User-ID: 999999	400	An ID with no matching user record must be rejected with 400.
8	String X-User-ID	X-User-ID: abc	400	The header value must be a parseable integer; a string must be rejected.
9	Admin endpoint without X-User-ID	X-Roll-Number only	200	Admin endpoints do not require X-User-ID and must remain accessible with only the roll number header.

Module 2: Admin Endpoints (test_admin.py)

These tests confirm that admin endpoints are accessible only with a valid roll number header and that they return the correct extended data sets.

#	Test Case	Input	Expected	Justification
10	GET all 7 admin endpoints	Valid X-Roll-Number	200 on all	All admin endpoints must be reachable and return 200 when the correct header is provided.
11	Admin without X-Roll-Number	No headers	401	Admin endpoints must also enforce the roll number requirement.
12	GET /admin/users	Valid headers	List with wallet + loyalty	The admin user list must return full user objects including wallet and loyalty data.
13	GET /admin/users/{valid_id}	Known user_id	Matching user object	A valid user ID must return the specific user record and nothing else.
14	GET /admin/users/999999	Non-existent ID	404	A non-existent user ID must produce a 404 response, not an empty 200.
15	Admin products vs user products	Both endpoints queried	Admin count >= user count	The admin endpoint must include inactive products, so its count cannot be less than the user-facing count.

Module 3: Profile (test_profile.py)

These tests cover GET and PUT operations on the user profile endpoint, including boundary value analysis on the name and phone fields.

#	Test Case	Input	Expected	Justification
16	GET /profile	Valid headers	200 + user data	Core profile retrieval must work and return the user's data.

#	Test Case	Input	Expected	Justification
17	GET /profile JSON structure	Valid headers	Contains user_id and name	The response must include at minimum the user_id and name fields as specified.
18	PUT valid name and phone	name=Test User, phone=9876543210	200	A correctly formed update request must be accepted.
19	PUT name = 1 char	name=A	400	Boundary below minimum: name must be at least 2 characters.
20	PUT name = 51 chars	name = 51 × A	400	Boundary above maximum: name must not exceed 50 characters.
21	PUT name = 2 chars (lower boundary)	name=AB	200	Exactly 2 characters is the minimum valid length and must be accepted.
22	PUT name = 50 chars (upper boundary)	name = 50 × A	200	Exactly 50 characters is the maximum valid length and must be accepted.
23	PUT phone = 9 digits	phone=123456789	400	One digit short of the required 10; must be rejected.
24	PUT phone = 11 digits	phone=12345678901	400	One digit over the required 10; must be rejected.
25	PUT phone with letters	phone=98765abcde	400	Phone must consist of digits only; alphabetic characters must be rejected.
26	PUT phone = 10 digits	phone=1234567890	200	Exactly 10 digits is the valid length and must be accepted.

Module 4: Addresses (test_addresses.py)

These tests validate the address creation and deletion endpoints, with particular focus on field-level boundary values for label, street, city, and pincode.

#	Test Case	Input	Expected	Justification
27	POST valid address [SERVER BUG]	HOME, pincode=500032	200	A valid 6-digit pincode must be accepted. This test fails due to inverted validation on the server.
28	POST label = WORK	label=WORK	400	Only HOME, OFFICE, and OTHER are valid labels; WORK must be rejected.
29	POST street = 3 chars	street=Abc	400	Street must be at least 5 characters; 3 is below the minimum.

#	Test Case	Input	Expected	Justification
30	POST street = 101 chars	street = 101 × A	400	Street must not exceed 100 characters; 101 is above the maximum.
31	POST city = 1 char	city=A	400	City must be at least 2 characters; 1 is below the minimum.
32	POST city = 51 chars	city = 51 × A	400	City must not exceed 50 characters; 51 is above the maximum.
33	POST pincode = 5 digits	pincode=50003	400	Pincode must be exactly 6 digits; 5 digits must be rejected.
34	POST pincode = 7 digits	pincode=5000321	400	Pincode must be exactly 6 digits; 7 digits must be rejected.
35	POST pincode with letters	pincode=5000AB	400	Pincode must be numeric; alphabetic characters must be rejected.
36	DELETE non-existent address	address_id=99999 9	404	Spec states that a non-existent address ID must return 404.

Module 5: Products (test_products.py)

These tests verify product listing, filtering, sorting, searching, and individual product retrieval against both the user-facing and admin endpoints.

#	Test Case	Input	Expected	Justification
37	GET /products returns 200	Valid headers	200	The core products endpoint must be reachable.
38	Products list is array	Valid headers	JSON array	The response body must be a JSON array, not an object or null.
39	Inactive products hidden	Compare admin vs user list	No inactive in user list	Products marked inactive must not appear in the user-facing product list.
40	Filter by category	?category=X	Only matching products	The category filter must return only products belonging to the specified category.
41	Sort by price ascending	?sort=price_asc	Non-decreasing prices	The sorted list must have no price that is lower than the preceding one.
42	Sort by price descending	?sort=price_desc	Non-increasing prices	The sorted list must have no price that is higher than the preceding one.

#	Test Case	Input	Expected	Justification
43	Search by name	?search=name	Matching products returned	The search parameter must filter results to products whose names contain the search term.
44	GET /products/{valid_id}	Known product_id	200 + product object	A valid product ID must return its full product object.
45	GET /products/999999	Non-existent ID	404	A non-existent product ID must return 404.
46	Price matches admin DB	Same product, both endpoints	Prices equal	The price returned to users must exactly match the admin record; no discrepancy is permitted.

Module 6: Cart (test_cart.py)

These tests cover adding, updating, removing, and clearing items in the cart, along with validation of quantity constraints and subtotal calculations.

#	Test Case	Input	Expected	Justification
47	Add with qty=0 [SERVER BUG]	quantity=0	400	The minimum quantity is 1; zero must be rejected. The server incorrectly accepts it.
48	Add with qty=-1	quantity=-1	400	Negative quantities are not meaningful and must be rejected.
49	Add non-existent product	product_id=999999	404	A product must exist before it can be added to the cart.
50	Add qty > stock	qty = stock + 100	400	The requested quantity cannot exceed what is available in stock.
51	Add valid item (qty=1)	qty=1, valid product	200	Standard happy-path addition to the cart must succeed.
52	Add same product twice	Add 1 then 1 same product	quantity=2 in cart	Adding the same product a second time must increment the existing quantity rather than create a duplicate line.
53	Item subtotal = qty × price	qty=2	subtotal = 2 × price	Each line's subtotal must be the exact arithmetic product of quantity and unit price.
54	Cart total = sum of subtotals	Multiple items in cart	total = sum of subtotals	The cart total must equal the sum of all individual line subtotals.
55	Update qty to 0	quantity=0 on update	400	The minimum quantity for an update is also 1; zero must be rejected.

#	Test Case	Input	Expected	Justification
56	Remove item not in cart	product_id not present	404	Attempting to remove a product that is not in the cart must return 404.
57	Clear cart	DELETE /cart/clear	Empty items list	After clearing, the cart must contain no items.

Module 7: Coupons (test_coupons.py)

These tests verify coupon validation logic, including rejection of invalid, expired, and threshold-failing codes, as well as correctness of percentage discount calculations.

#	Test Case	Input	Expected	Justification
58	Apply non-existent coupon	code=INVALIDCODE999	400 or 404	An unrecognised coupon code must be rejected; it cannot apply any discount.
59	Apply expired coupon	Expired coupon code	400 or 422	Expired coupons must be rejected regardless of the discount amount.
60	Apply below minimum cart value	High-min coupon, low total	400 or 422	A coupon requiring a minimum cart value must be rejected if the cart total falls short.
61	PERCENT coupon discount maths	PERCENT coupon applied	discount = total × % / 100	The percentage discount must be calculated correctly and match the arithmetic result.
62	Remove coupon	POST /coupon/remove	200 or 400	Removing a coupon must not crash the server; a clean response is required in both the applied and not-applied states.

Module 8: Checkout (test_checkout.py)

These tests validate the checkout flow across all three payment methods, including payment status initialisation, the COD order limit, and GST calculation.

#	Test Case	Input	Expected	Justification
63	Checkout empty cart	No items in cart	400	Spec states that an empty cart must return 400; no order should be created.
64	Invalid payment method	payment_method=BITCOIN	400	Only COD, WALLET, and CARD are valid methods; anything else must be rejected.
65	COD with total > 5000	Large cart + COD	400	Cash on delivery has a 5000 limit; orders exceeding it must be blocked.

#	Test Case	Input	Expected	Justification
66	CARD payment status	CARD checkout	payment_status =PAID	CARD payments are settled immediately and must start in PAID status.
67	COD payment status	COD checkout, total <= 5000	payment_status =PENDING	COD payments are collected at delivery and must start in PENDING status.
68	GST is 5%	Invoice after checkout	gst = subtotal × 0.05	The GST rate must be exactly 5%; any deviation indicates a calculation error.
69	Invoice total = subtotal + GST	Invoice data	total = subtotal + gst	The invoice total must exactly equal the sum of the subtotal and the calculated GST.

Module 9: Wallet (test_wallet.py)

These tests cover top-up and payment operations on the wallet, including boundary values on the top-up limit and validation that deductions are exact.

#	Test Case	Input	Expected	Justification
70	Add amount=0	amount=0	400	Amount must be greater than zero; zero is not a valid top-up.
71	Add negative amount	amount=-100	400	Negative amounts are meaningless for a wallet top-up and must be rejected.
72	Add amount=100001	amount=100001	400	The single top-up limit is 100,000; one unit above must be rejected.
73	Add amount=100000 (boundary)	amount=100000	200	Exactly 100,000 is the maximum valid top-up amount and must be accepted.
74	Add increases balance	amount=500	Balance increases by 500	The wallet balance must increase by the exact top-up amount.
75	Pay amount=0	amount=0	400	Zero is not a valid payment amount and must be rejected.
76	Pay > balance	amount > current balance	400	Payments exceeding the available balance must be blocked.
77	Pay deducts exact amount [SERVER BUG]	amount=100	Balance decreases by 100	Spec requires exact deduction. The server incorrectly deducts 100.80, losing the user 0.80 extra.

Module 10: Loyalty (test_loyalty.py)

These tests verify the loyalty points balance endpoint and the redemption validation rules.

#	Test Case	Input	Expected	Justification
78	GET /loyalty	Valid headers	200	Basic retrieval of the loyalty balance must succeed.
79	Response has points field	Valid headers	Contains points field	The response must include the points field as specified.
80	Redeem 0 points	points=0	400	The minimum redeemable amount is 1 point; zero must be rejected.
81	Redeem negative points	points=-10	400	Negative redemption amounts are invalid and must be rejected.
82	Redeem more than balance	points > current balance	400	Cannot redeem more points than are available in the account.

Module 11: Orders (test_orders.py)

These tests cover order listing, single-order retrieval, cancellation constraints, stock restoration on cancellation, and invoice arithmetic.

#	Test Case	Input	Expected	Justification
83	GET /orders	Valid headers	200	The orders list endpoint must be reachable and return a response.
84	Orders list is array	Valid headers	JSON array	The response body must be a JSON array.
85	GET /orders/999999	Non-existent order_id	404	A non-existent order ID must return 404.
86	Cancel non-existent order	order_id=999999	404	Spec states that a non-existent order must return 404.
87	Cancel DELIVERED order	Delivered order_id	400	A delivered order cannot be cancelled; the server must reject the request with 400.
88	Cancel valid — stock restored	Place, cancel, verify stock	stock increases by qty	Cancelling an order must return the reserved stock to the available quantity.
89	Invoice: subtotal + GST = total	Invoice data	total = subtotal + gst	Invoice totals must be arithmetically consistent.
90	Invoice: GST = 5%	Invoice data	gst = subtotal × 0.05	The GST rate on order invoices must be exactly 5%.

Module 12: Reviews (test_reviews.py)

These tests validate rating boundary values, comment length constraints, and the correctness of average rating calculations.

#	Test Case	Input	Expected	Justification
91	Rating = 0	rating=0	400	Rating must be at least 1; zero is below the minimum.
92	Rating = 6	rating=6	400	Rating must not exceed 5; 6 is above the maximum.
93	Rating = -1	rating=-1	400	Negative ratings are invalid and must be rejected.
94	Rating = 1 (lower boundary)	rating=1	200	1 is the minimum valid rating and must be accepted.
95	Rating = 5 (upper boundary)	rating=5	200	5 is the maximum valid rating and must be accepted.
96	Empty comment	comment=empty string	400	A comment must be at least 1 character; an empty string is invalid.
97	Comment = 201 chars	comment = 201 × A	400	Comment must not exceed 200 characters; 201 is above the maximum.
98	Comment = 200 chars (boundary)	comment = 200 × A	200	200 characters is the maximum valid length and must be accepted.
99	Average rating decimal [SERVER BUG]	Ratings 4 and 3	avg = 3.5	Spec requires proper decimal calculation. The server incorrectly returns 3 (integer floor).
100	No reviews → avg = 0	Product with no reviews	average_rating = 0	When no reviews exist the default average must be 0 as specified.

Module 13: Support Tickets (test_tickets.py)

These tests cover ticket creation with boundary values on subject and message, and enforce the correct forward-only status transition rules.

#	Test Case	Input	Expected	Justification
101	Create valid ticket	subject 5–100 chars, message 1–500 chars	200	A well-formed ticket creation request must succeed.
102	New ticket status = OPEN	Create ticket	status=OPEN	A newly created ticket must always start in OPEN status.
103	Subject = 2 chars	subject=Hi	400	Subject must be at least 5 characters; 2 is below the minimum.

#	Test Case	Input	Expected	Justification
104	Subject = 101 chars	subject = 101 × A	400	Subject must not exceed 100 characters; 101 is above the maximum.
105	Subject = 5 chars (lower boundary)	subject=Hello	200	Exactly 5 characters is the minimum valid subject length and must be accepted.
106	Subject = 100 chars (upper boundary)	subject = 100 × A	200	Exactly 100 characters is the maximum valid subject length and must be accepted.
107	Empty message	message=empty string	400	Message must be at least 1 character; an empty message must be rejected.
108	Message = 501 chars	message = 501 × A	400	Message must not exceed 500 characters; 501 is above the maximum.
109	Message saved exactly [SERVER BUG]	Special character message	Stored message = input	The server must preserve the full message including special characters. This test fails — the message is truncated or altered.
110	OPEN → IN_PROGRESS	status=IN_PROGRESS	200	Transitioning from OPEN to IN_PROGRESS is a valid forward move and must be accepted.
111	IN_PROGRESS → CLOSED	status=CLOSED	200	Transitioning from IN_PROGRESS to CLOSED is the correct final step and must be accepted.
112	OPEN → CLOSED [SERVER BUG]	status=CLOSED from OPEN	400	Tickets must pass through IN_PROGRESS before closing. The server incorrectly allows skipping this step.
113	CLOSED → OPEN [SERVER BUG]	status=OPEN from CLOSED	400	Status transitions must be forward-only; reverting a CLOSED ticket must be blocked. The server allows it.
114	Invalid status PENDING [SERVER BUG]	status=PENDING	400	Only OPEN, IN_PROGRESS, and CLOSED are valid statuses. The server accepts and stores arbitrary strings.

3. Bug Report

The following bugs were confirmed by running the full test suite against the live QuickCart server. All failures were reproducible. Each report includes the endpoint, request details, the expected behaviour per the API specification, the observed behaviour, and a severity rating.

Overall test run: 82 passed | 16 failed | 20 skipped

Bug	Endpoint	Method	Request / Headers	Expected	Actual	Severity
#1	POST /api/v1/addresses	POST	X-Roll-Number + X-User-ID; body: {label:HOME, street:123 Test Street, city:Hyderabad, pincode:500032, is_default:false}	200 — valid 6-digit pincode accepted	400 — Invalid pincode. Server accepts 5- and 7-digit pincodes but rejects exactly 6 digits. Logic is inverted.	High
#2	GET /api/v1/profile	GET	X-Roll-Number + X-User-ID: 999999	400 — spec says invalid X-User-ID returns 400	404 — User not found. Functionally reasonable but violates the spec's status code contract.	Low
#3	POST /api/v1/cart/add	POST	X-Roll-Number + X-User-ID; body: {product_id: <valid>, quantity: 0}	400 — minimum quantity is 1	200 — Item added to cart. Zero-quantity items corrupt cart totals.	High
#4	PUT /api/v1/profile	PUT	X-Roll-Number + X-User-ID; body: {name:Test User, phone:98765abcde}	400 — phone must be 10 digits only	200 — Profile updated successfully. Malformed phone number stored.	Medium
#5	GET /api/v1/products/{id}/reviews	GET	X-Roll-Number + X-User-ID; set of ratings averaging 3.2	average_rating: 3.2 (decimal)	average_rating: 3 (integer floor). Misleading rating shown to users.	Medium
#6	GET /api/v1/support/tickets	GET	X-Roll-Number + X-User-ID	Each ticket includes the full message field	Ticket objects only return ticket_id, subject, status — message field is absent.	Medium
#7	PUT /api/v1/support/tickets/{id}	PUT	X-Roll-Number + X-User-ID; body: {status:CLOSED} on an OPEN ticket	400 — must transition through IN_PROGRESS first	200 — Ticket updated. OPEN→CLOSED skip accepted; workflow integrity broken.	High

Bug	Endpoint	Method	Request / Headers	Expected	Actual	Severity
#8	PUT /api/v1/support/tickets/{id}	PUT	X-Roll-Number + X-User-ID; body: {status:OPEN} on a CLOSED ticket	400 — status is forward-only	200 — Ticket updated. CLOSED tickets can be re-opened arbitrarily.	High
#9	PUT /api/v1/support/tickets/{id}	PUT	X-Roll-Number + X-User-ID; body: {status:PENDING}	400 — only OPEN, IN_PROGRESS, CLOSED valid	200 — Ticket updated with status=PENDING stored in DB. No input sanitisation.	High
#10	POST /api/v1/wallet/pay	POST	X-Roll-Number + X-User-ID; body: {amount: 100}	Balance decreases by exactly 100	Balance decreases by 100.80. User loses an unauthorised 0.80 per transaction.	Critical

Tests run against QuickCart x86 Docker image